

## Model Context Protocol (MCP)

How things worked before MCP -

example : AI agent --- today's weather.

user : what is the weather in delhi today?

```
✓ weather = requests.get("https:// - ")
```

```
✓ data = weather.json()
```

```
✓ print (data ["temperature"])
```

o/p - sunny in Delhi with 22°C temperature.

→ Weather Company - API modified

"temperature" → "temp"

then,

weather = \_\_\_\_\_

data = \_\_\_\_\_

X won't work

print ( \_\_\_\_\_ )

↓ changed to "temp"

print (data ["temp"]) ✓

We need change according to Company changes.

→ how MCP works

MCP Client

↓  
has agent

company infrastructure  
MCP Server

↓  
weather MCP server

\* User query ↔ MCP Client ↔ MCP Server ↔ Company API

Every company has MCP servers → All MCP servers communicate using JSON-RPC

→ Why MCP? if company ever change any api codes, we need not worry cause servers will be updated.

MCP Client need not do any modifications.

→ Weather Company - API modified

"temperature" → "temp"

then,

weather = ~

data = ~

X won't work

print (~[~])

↙ changed to "temp"

print (data ["temp"]) ✓

We need change according to Company changes.

→ how MCP works

MCP Client

↓  
has agent

company infrastructure  
MCP Server

↓  
weather MCP server

\* User query → MCP Client ↔ MCP Server ↔ Company API

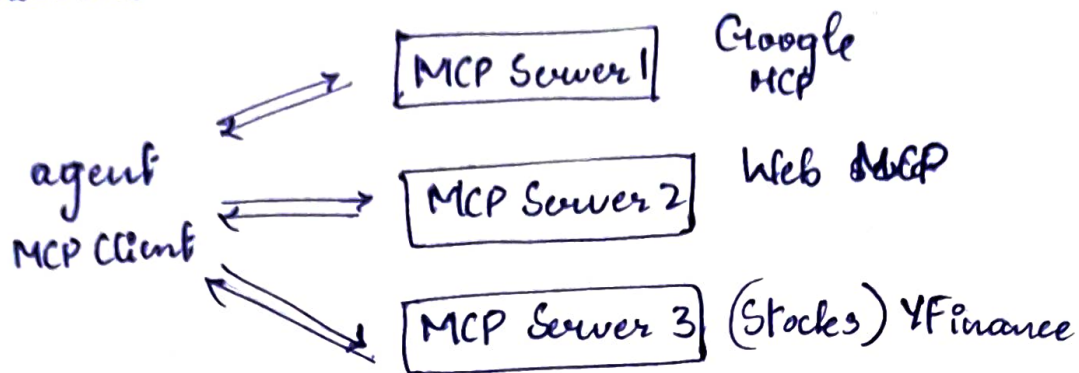
Every company has MCP servers → All MCP servers communicate using JSON-RPC

→ Why MCP? if company ever change any api codes, we need not worry cause servers will be updated.

MCP Client need not do any modifications.

## \* Benefits of working with MCP -

### 1. Universal Communication Standard:



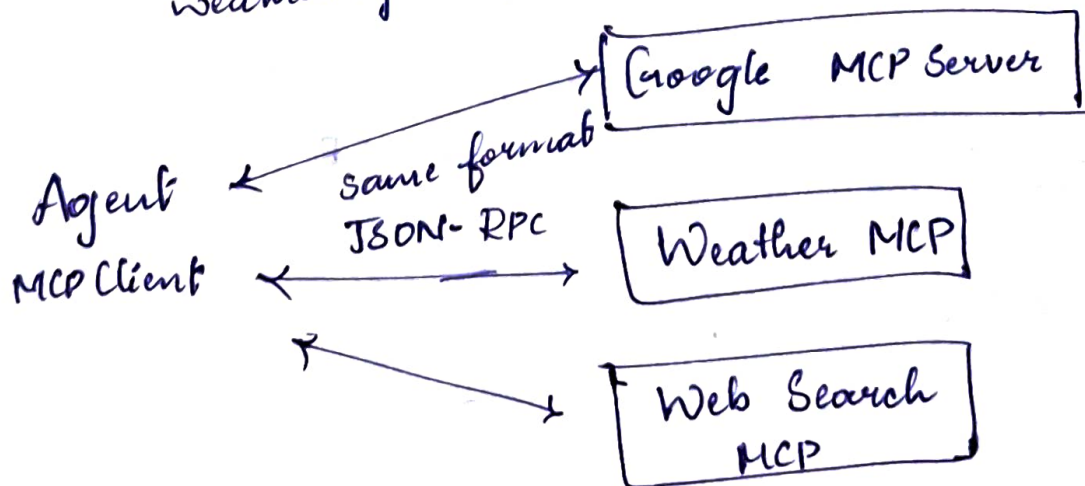
We can connect to multiple companies MCP servers through agent (one) and it needs to have MCP Client.

### 2. No code changes when backend's change

### 3. Works with any MCP Compatible tool -

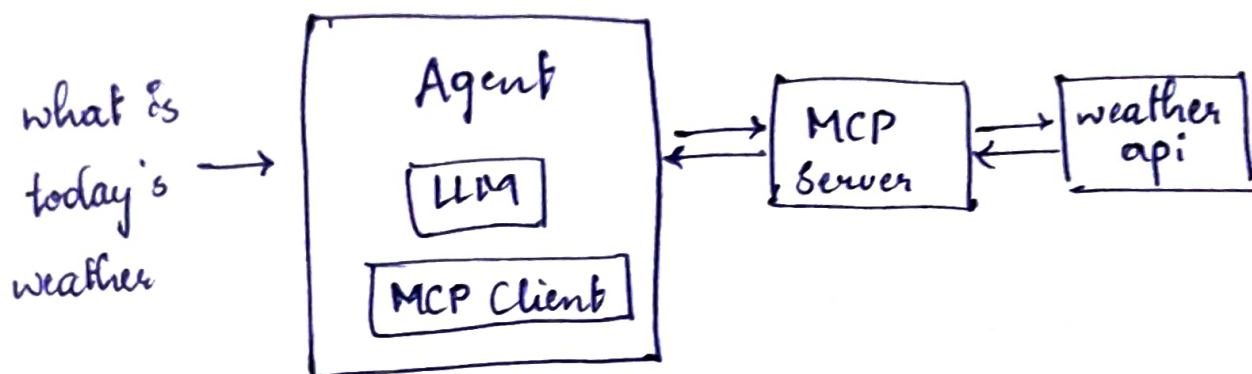
example : user : Send me an email about gmail api

today's weather and top 3 news headlines.  
weather api news api



## \* Building and Linking multiple MCP Servers

↳ Run a single MCP Server - AI agent using  
LangGraph → Weather Server



Need to go and visit MCP git repo by search  
and clone the repo in local machine.

↳ Pre requisites -

⇒ `go get go.dev/cmd/`

install and setup the app.

add path to environment variables.

⇒ get api-key of weather

→ import asyncio → no idle time while loading

import os

from dotenv import load\_dotenv

from langchain\_openai import ChatOpenAI

" " - mcp - adapters . client import

MultiServer MCPClient

" " . graph import StateGraph, START,  
MessageState

from langgraph.prebuilt import ToolNode,

tools - condition

→ Client = MultiServer MCPClient{

```
{
  "weather": {
    "transport": "stdio",
    "command": "E: / --", # path
    "args": [],
    "env": {"OWM_API_KEY": own-key}
  }
}
```

```
{
  tools = await client.get_tools()
  model = ChatOpenAI(model=" ", api-key= --)
  & bind em.
```

→ def call\_model(state: MessageState):

```
    response = model.bind_tools(tools).invoke
        (state["messages"])

    return {"messages": response}
```

→ # building workflow

```
builder = StateGraph(MessagesState)
```

```
builder.add_node("call_model", call_model)
```

```
    ("tools", ToolNode(tools))
```

```
builder.add_edge(START, "call_model")
```

```
    .add_conditional_edge("call_model", tools_condition)
```

```
builder.add_edge("tools", "call_model")
```

```
graph = builder.compile()
```

```
print("In -- Weather Query --")
```

# meaning the graph  
result = await graph.alive (2 "messages" :

"What's weather in Delhi"?)

```
print (result ["messages"] [-1].content)
```

2 terminals to run

→ ⇒ • \mcp-weather.exe # MCP server

other one  $\Rightarrow$  python l.py

```
if __name__ == "__main__":
    asyncio.run(main())
```

\* This was Single MCP Server, let's see multi  
MCP servers. # same code.  
just changing the Client.

Client = MultiServerMCPClientC

9 "weather": 2

"calculator": {

"any other": { ~~~~~ }

30 - 2

# 3 command prompts to run 2 servers and our app. Go to git for local servers

# \* Creating a Custom MCP Server with LangGraph

22/08/26

## and Streamlit

Why custom?

- provide functionality no existing MCP server offers.
- keep private company data secure.

Setup the environment -

- build an app with LangGraph + Streamlit.
- connect it to our custom MCP server

pip install mcp

### Practical

from mcp-server.fastmcp import FastMCP

mcp = FastMCP("Math")

@mcp.tool() # decorator to make it a tool.

def add(a: int, b: int) → int:  
 return a + b

# Similarly other functions

@mcp.tool()

def multiply(—):

@mcp.tool()

def divide(—):

@mcp.tool()

def sq-root(—):

@mcp.tool()

def factorial(—):



```

async def call_model(state: MessagesState):
    messages = state["messages"]
    response = await model_with_tool.ainvoke
        (messages)
    return {"messages": [response]}

```

```

builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
builder.add_node("tools", tool_node)
builder.add_edge(START, "call_model")
builder.add_conditional_edges("call_model",
    should_continue,)

```

```

builder.add_edge("tools", "call_model")

```

```

# compile the graph

```

```

graph = builder.compile()

```

```

# running the graph

```

```

result = await graph.ainvoke({"messages": "what's (3+5) x 12?"})

```

```

print(result["messages"][-1].content)

```

```

if __name__ == "__main__":
    asyncio.run(main())

```

~~if~~ If the MCP server is running remotely at other locations then in `def main():`,

we just need to change :

```
→ client = Mocket { "math" : {  
    "transport": "streamable-http",  
    "url": "#url of mcp server",  
  }  
}
```

and in mcp-server.py file where we have tools:

```
if __name__ == "__main__":  
    # mcp.run (transport="stdio")  
    mcp.run (transport="streamable-http")
```

## \* Build a Custom MCP Server on the Cloud.

- Setup a new VPS
- install req packages
- custom MCP server code on VPS
- run server on cloud
- connect local MCP client using langGraph

/hostinger - purchase it

we use VPS dashboard

Putty configuration — pwd enter (used to connect to cloud)

later install packages after creating venv.

• Previous mcp server code we have send it to this.

• open powershell and cd # directory path

• To send this cloud :

pscp custom-mcp-s.py root@31.97.233.200/opt/mcp-server  
file in local machine → pasting it here in this file

\* In PuTTY -

python custom-server.py

our server is running.

\* Like earlier, same code in <sup>client</sup> ~~server~~ just change

client = M — ( —

"url" : "http:// — /mcp"

↓  
ip address of our cloud

\* In server, we give :

if ~~name~~ host = os.getenv("HOST", "0.0.0.0")

port = int ( <sup>1</sup> " ("PORT", "8080")

log.info ( f " — " )

app = mcp.streamable-http-app ( )

unicorn.run (app, host = host, port = port)

## \* MCP Servers with Agno

- ↳ single & multiple servers
- ↳ custom MCP server
- ↳ local & cloud MCPs

⇒ Calculator one from git - installation.

### Server file:

```
→ import asyncio
    " os
    from dotenv
    from agno.tools.mcp import MCPTools
    " agno.agent import Agent
    " agno.models.openai from OpenAIChat
```

<sup>a</sup>  
load\_dotenv()

async def main():

openai\_key = \_\_\_\_\_

agent = Agent(

model = \_\_\_\_\_

tools = [MCPTools("Python -m mcp\_server-  
calculator")

) description = " " , markdown = True

agent.print\_response("What is 12<sup>th</sup> 8?", stream = True)

```
if __name__ == "__main__":
    asyncio.run(main())
```

\* run it, will work

\* For multiple servers, install em 1<sup>st</sup> then in code:

```
→ agent = Agent(model = _____,
                  tools = [
                      MCPTools("_____"),
                      MCPTools("_____"),
                      MCPTools("_____"),
                      ],
                  description = "_____",
                  markdown = True
                  )
```

↳ from the servers

\* For custom, code for custom server, write code like math-(add, ~) like previous examples. For this, no need to run the server the client code will run it automatically.

Client file:

```
→ agent = Agent(model = _____,
                  tools = [
                      MCPTools("_____"),
                      MCPTools("_____"),
                      MCPTools("python math_server.py")
                      ],
                  )
```

↳ custom one

\* For Cloud MCP Server - (remote)

Same steps to follow, run virtual env and run it like earlier.

Client file :

→ # imports

# load env

math\_saver\_url = "http://www.mathsaver.com"

your vps with server (remote)

```
async def run-agent (message: str):
```

multi-ncp-tools = MultiMCPTools (

```
commands = [],
```

```
urls = [math_server_url],
```

```
urls_transports = ["streamable-http"],
```

```
await multi_mcp_tools.connect()
```

agent - Agent (model =           )

tools> [matlab, mcp-tools],

$$d_{\text{eff}} = \frac{1}{\sqrt{2}}$$

mark down: True,

```
print ("In Agent Response: In")
```

```
print ("In Agent Response: in")
await agent.aprint_response(msg, st)
```

await multi-tools.close()

if ———

```
asyncio.run(run_agent("Query"))
```